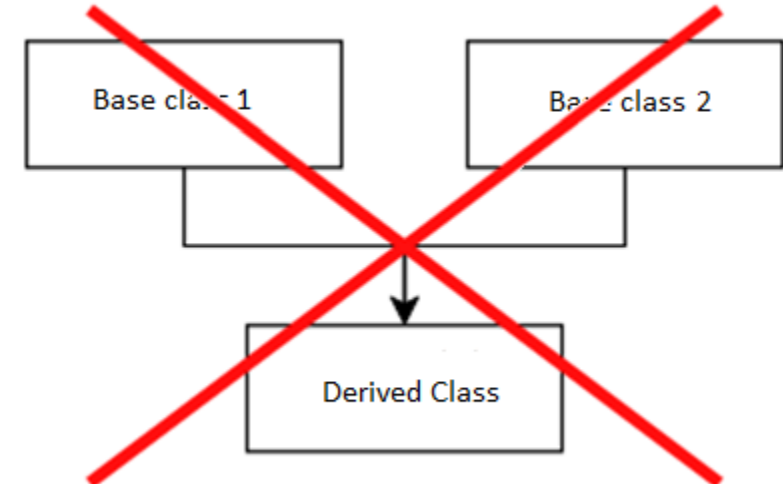
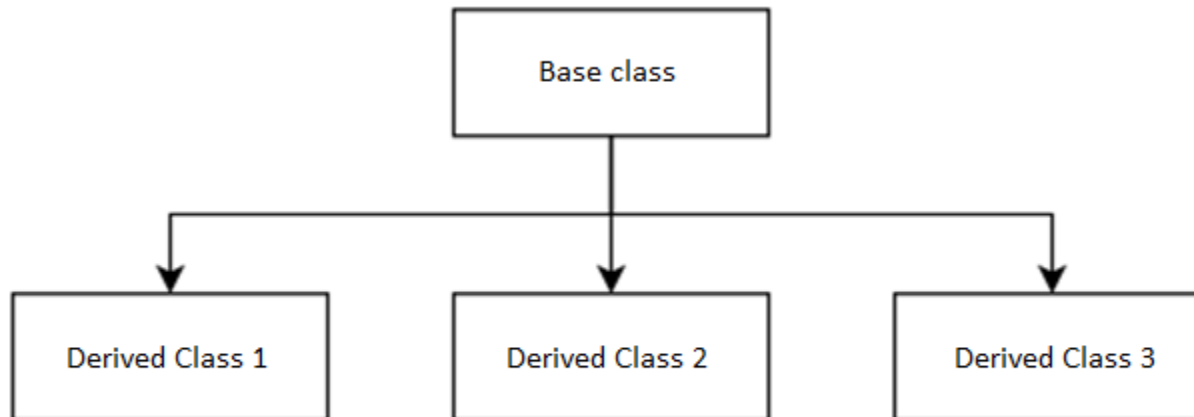


Inheritance and access modifiers

Inheritance (Derivation)

- Classes can inherit from each other
- inherit properties and methods of the base class*
- It can be inherited by any number of classes
- It can be inherited by only one class at a time



*not all members are taken over (e.g. no constructor/destructor)

Constructors in derived classes

- when initializing a new instance, the constructor of the base class is always executed first
- if the constructor of the base class contains parameters, these must be explicitly specified with ": base(Parameter)".

```
public class Species
{
    public string Name { get; set; }

    public Species(string name)
    {
        this.Name = name;
    }
}

public class Human : Species
{
    public int Age { get; set; }

    public Human(string name, int age) : base(name)
    {
        this.Age = age;
    }
}
```

virtual members

- behave like normal members and can also be called like that
- can be overridden by a derived class with the keyword "override"
- with keyword "base" the base class member can be accessed

```
public class Species
{
    public virtual string WhatAmI()
    {
        return "I am a species";
    }
}

public class Human : Species
{
    public override string WhatAmI()
    {
        string whatami = base.WhatAmI();
        whatami += " with two legs and I can speak";
        return whatami;
    }
}
```

Prevent override

- you can overwrite virtual members continuously, even if a class is derived from an already derived class
- this can be connected by using the keyword "sealed" and thus preventing the newly deriving classes from overwriting

```
public class Species
{
    public virtual string WhatAmI()
    {
        return "I am a species";
    }
}

public class Human : Species
{
    public sealed override string WhatAmI()
    {
        string whatami = base.WhatAmI();
        whatami += " with two legs and I can speak";
        return whatami;
    }
}

public class Child : Human
{
    //Can't be overridden
    public override string WhatAmI()
    {
        return "I am a child";
    }
}
```

Access modifier

- can be used for fields, properties, methods and whole classes

Modifier	Access
public	from outside the class
private	only within the class
protected	within the class and in all derived classes
internal	only the current assembly
protected internal	only the current assembly or derived class
private protected	only derived classes within the assembly